

COL7(6)83 Assignment 4

Morphological Operations, Edge Detection, and Segmentation

November 16, 2025

1 Problem 1: Text Character Detection using Binary Morphology

1.1 Overview

This problem explores binary morphological operations for detecting specific characters in scanned text documents. Starting with a thresholded binary image of printed text, we progressively refine our detection strategy to identify all instances of the lowercase letter "c" while avoiding false positives from similar characters.

The input image A is a binary representation where text pixels have value 1 and background pixels have value 0. A manually cropped lowercase "c" serves as the initial structuring element B .

1.2 Part (a): Basic Morphological Operations

1.2.1 Implementation

Binary erosion and dilation form the foundation of mathematical morphology. For a binary image A and structuring element B , these operations are defined as:

Dilation:

$$(A \oplus B)(i, j) = \bigvee_{(s, t) \in B} A(i + s, j + t)$$

The dilation operation places the structuring element at each foreground pixel in A and marks all positions covered by B as foreground in the output. This effectively expands the foreground regions.

Erosion:

$$(A \ominus B)(i, j) = \bigwedge_{(s, t) \in B} A(i + s, j + t)$$

The erosion operation places the structuring element at each position and marks the center as foreground only if B fits entirely within the foreground of A . This shrinks foreground regions.

Closing:

$$A \bullet B = (A \oplus B) \ominus B$$

Closing performs dilation followed by erosion, which fills small gaps and connects nearby components while approximately preserving their size.

1.2.2 Results

We demonstrate dilation and closing on the text image A using a disk-shaped structuring element with radius $r = 2$ pixels. The disk is defined as:

$$\text{disk}(r) = \{(x, y) : x^2 + y^2 \leq r^2\}$$

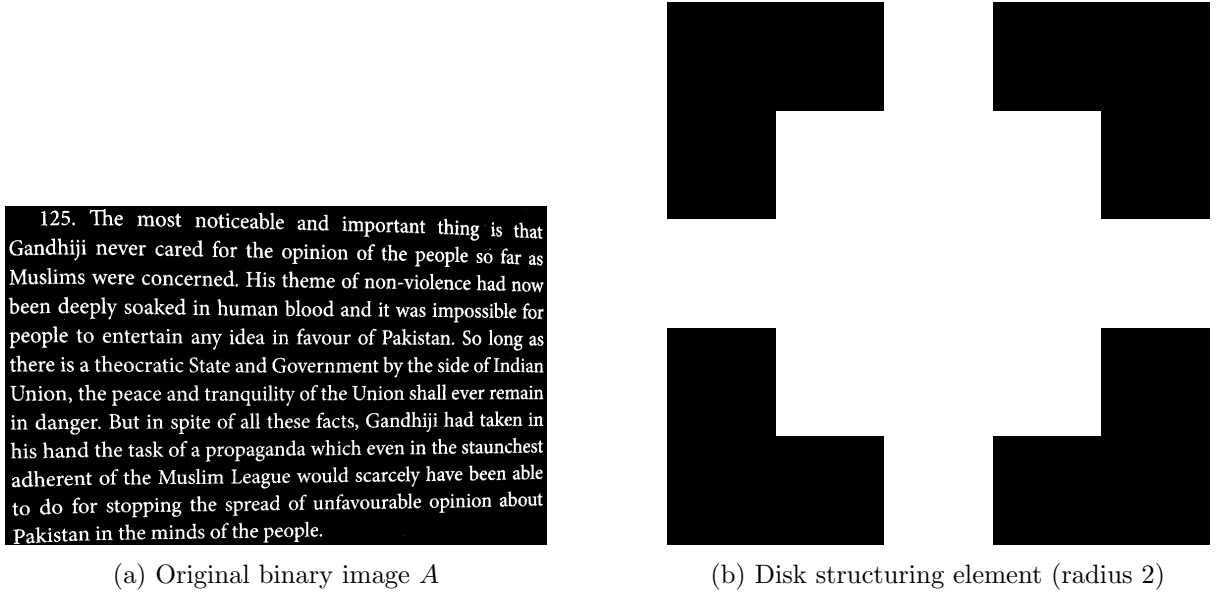


Figure 1: Input image and structuring element

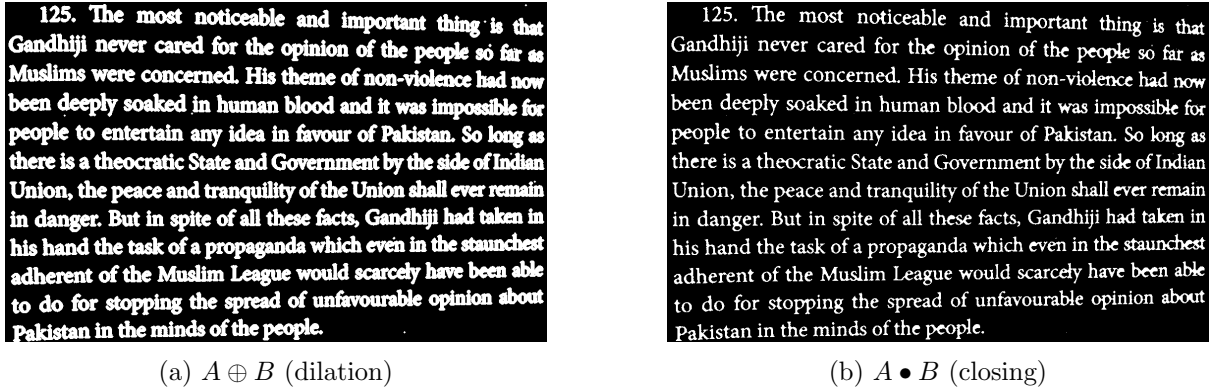


Figure 2: Results of dilation and closing operations

The dilation expands all text strokes uniformly, making thin letters thicker. The closing operation fills small gaps within and between characters, bridging broken strokes while maintaining overall character structure.

1.3 Part (b): Initial Character Detection via Opening

1.3.1 Method

To detect occurrences of the letter "c", we use morphological opening:

$$A \circ B = (A \ominus B) \oplus B$$

Opening performs erosion followed by dilation. The erosion removes all regions where B does not fit, leaving only locations that match the shape of B . The subsequent dilation restores the remaining regions to approximately their original size.

For character detection, opening identifies locations where the structuring element B (the template "c") fits within the image A . Pixels that survive the erosion-dilation sequence indicate potential matches.

1.3.2 Results and Discussion

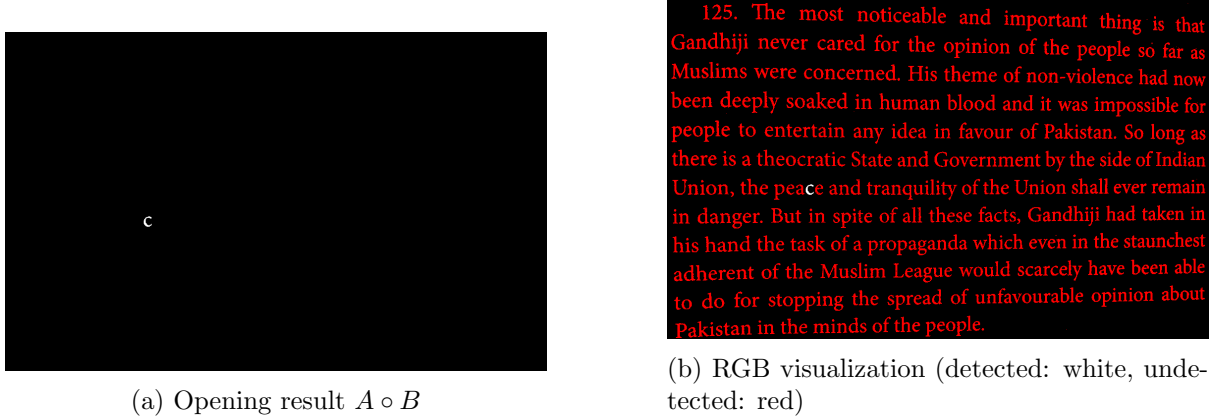


Figure 3: Part (b): Character detection using the original structuring element

The RGB visualization displays detected letters in white (all color channels active) and undetected letters in red (only red channel active). This clearly shows which instances of "c" were successfully identified.

Why not all c's were found: The opening operation is highly sensitive to exact shape matching. Several factors cause missed detections:

- **Font variations:** Even within the same font, individual character instances may have slight rendering differences due to anti-aliasing, printing artifacts, or scanning noise.
- **Size variations:** Small differences in character size prevent perfect alignment between B and target characters.
- **Stroke width variations:** Inconsistent ink density during printing or thresholding creates characters with varying stroke widths.
- **Structural requirements:** The erosion step requires B to fit completely within the foreground. Any pixel mismatch causes the entire character to be rejected.

The strict geometric requirements of erosion make opening unsuitable for robust character detection when exact pixel-perfect matching cannot be guaranteed.

1.4 Part (c): Improved Detection with Modified Structuring Element

1.4.1 Strategy

To detect all instances of "c", we must relax the strict geometric constraints while maintaining shape selectivity. The key insight is that the defining characteristic of "c" is its overall shape, not its exact pixel configuration.

We create a modified structuring element B_1 by eroding the original B with a small disk:

$$B_1 = B \ominus \text{disk}(1) \ominus \text{disk}(1)$$

The double erosion with unit disks shrinks B uniformly, reducing it to its "core" shape. This smaller structuring element is more tolerant to minor variations in character rendering.

1.4.2 Mathematical Rationale

The erosion process removes the boundary pixels of B , creating a smaller template that captures the essential topology of "c" without requiring exact boundary matching. This can be understood through the property:

$$(A \ominus B_1) \supseteq (A \ominus B)$$

Since $B_1 \subset B$, the smaller structuring element fits more easily, detecting a superset of the original matches. The trade-off is that B_1 may also match other letters with similar core shapes, such as "e".

1.4.3 Results

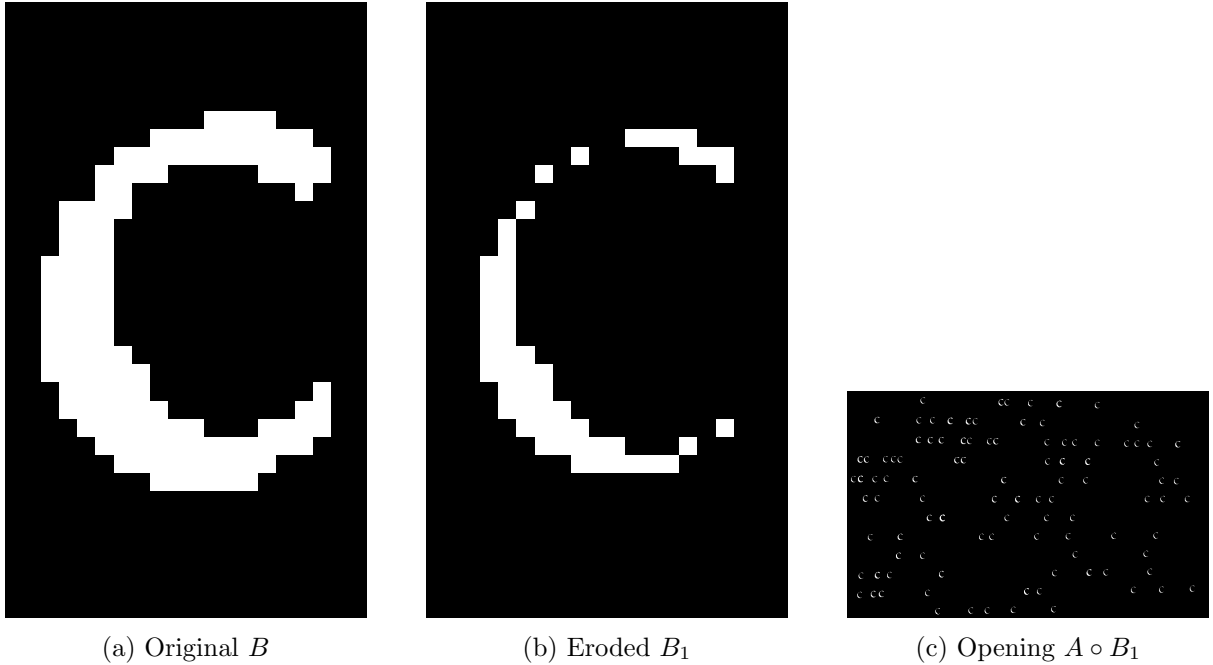


Figure 4: Modified structuring element and detection results

125. The most noticeable and important thing is that Gandhiji never cared for the opinion of the people so far as Muslims were concerned. His theme of non-violence had now been deeply soaked in human blood and it was impossible for people to entertain any idea in favour of Pakistan. So long as there is a theocratic State and Government by the side of Indian Union, the peace and tranquility of the Union shall ever remain in danger. But in spite of all these facts, Gandhiji had taken in his hand the task of a propaganda which even in the staunchest adherent of the Muslim League would scarcely have been able to do for stopping the spread of unfavourable opinion about Pakistan in the minds of the people.

Figure 5: RGB visualization showing improved detection (all c's detected in white)

The modified structuring element B_1 successfully detects all instances of "c" in the text. However, as predicted, it also detects some lowercase "e" characters, which share a similar core shape (a curved body with an opening on the right side). This demonstrates the fundamental trade-off in template matching: reducing selectivity to improve recall (finding all true positives) inevitably increases false positives.

1.5 Part (d): Precise Detection via Hit-or-Miss Transform

1.5.1 Theory

To eliminate false positives while maintaining complete detection, we employ the hit-or-miss transform, which tests for both the presence and absence of specific features. Given two structuring elements B_1 (hit template) and B_2 (miss template), the transform is defined as:

$$A \circledast (B_1, B_2) = (A \ominus B_1) \cap (A^c \ominus B_2)$$

where A^c denotes the complement of A . This operation identifies locations where:

1. The foreground structure matches B_1 (the "hit" condition)
2. The background structure matches B_2 (the "miss" condition)

1.5.2 Constructing the Miss Template

The key to distinguishing "c" from "e" lies in the gap region. In a lowercase "c", the right side is open with no horizontal bar crossing the middle. In contrast, "e" has a horizontal stroke in this region.

We construct B_2 to detect background pixels in the region where "e" would have its horizontal crossbar:

$$B_2 = \text{rectangular region} \times \text{disk}(1)$$

The rectangular region spans the middle height of the character and extends into the right-side opening. The dilation by a small disk provides tolerance for minor position variations.

1.5.3 Implementation Details

The complete detection procedure:

1. Erode A with B_1 to find locations matching the core "c" shape
2. Erode A^c (background) with B_2 to find locations with the required background pattern
3. Intersect these results to obtain points satisfying both conditions
4. Dilate the detected points with the original B to reconstruct full character shapes

The final dilation step is crucial for visualization, as the hit-or-miss transform outputs only single marker points rather than full character shapes.

1.5.4 Results

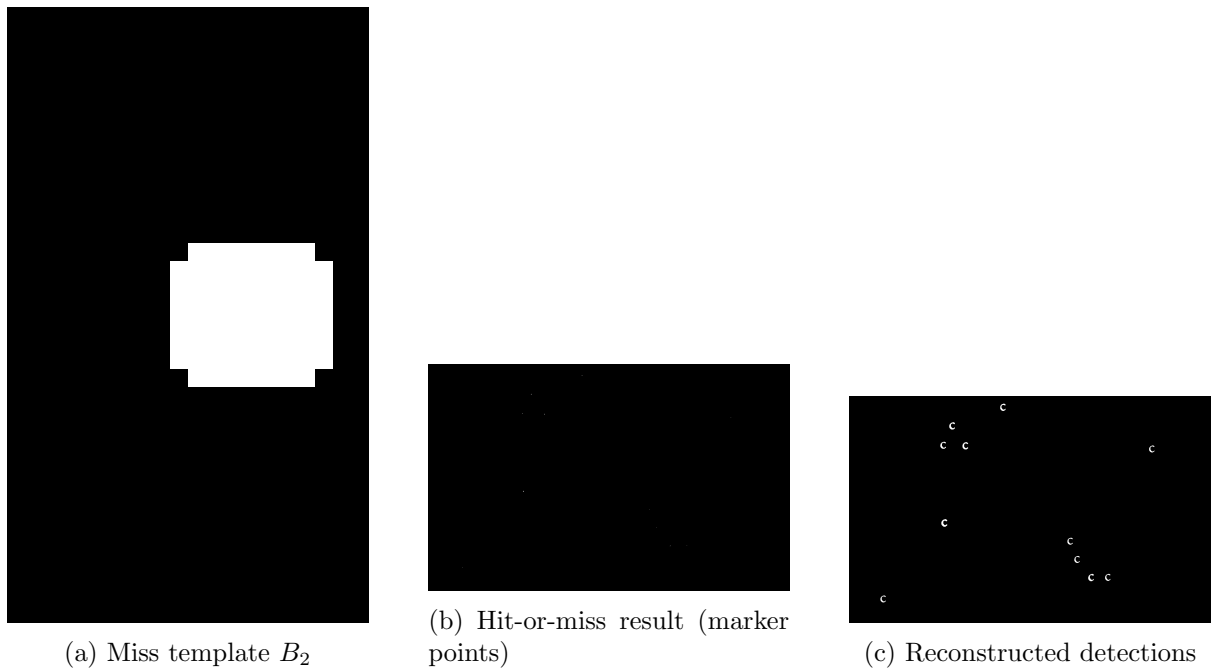


Figure 6: Hit-or-miss transform for precise "c" detection

125. The most noticeable and important thing is that Gandhiji never cared for the opinion of the people so far as Muslims were concerned. His theme of non-violence had now been deeply soaked in human blood and it was impossible for people to entertain any idea in favour of Pakistan. So long as there is a theocratic State and Government by the side of Indian Union, the peace and tranquility of the Union shall ever remain in danger. But in spite of all these facts, Gandhiji had taken in his hand the task of a propaganda which even in the staunchest adherent of the Muslim League would scarcely have been able to do for stopping the spread of unfavourable opinion about Pakistan in the minds of the people.

Figure 7: Final RGB visualization showing only "c" characters detected (white)

The hit-or-miss transform successfully identifies all instances of "c" while rejecting "e" characters. The miss template B_2 ensures that only characters with an open right side are detected. The marker points from the transform are then dilated by the original structuring element B to visualize the complete character shapes.

This approach demonstrates the power of morphological methods for shape-based pattern recognition: by combining positive (hit) and negative (miss) constraints, we achieve both high recall (finding all targets) and high precision (avoiding false positives).

2 Problem 2: Blood Vessel Analysis in Fundus Images

2.1 Overview

Fundus photography captures the interior surface of the eye, revealing the retinal blood vessel network. Analyzing vessel structure is crucial for diagnosing various ocular and systemic diseases. This problem develops a morphological pipeline to extract vessels from a fundus image and characterize their thickness distribution.

2.2 Part (a): Grayscale Morphological Operations

2.2.1 Implementation

Grayscale morphology extends binary operations to intensity images. For a grayscale image f and flat structuring element B , the operations are defined using supremum and infimum:

Grayscale Erosion:

$$(f \ominus B)(x, y) = \min_{(s, t) \in B} f(x + s, y + t)$$

At each position, erosion selects the minimum intensity within the structuring element's neighborhood. This darkens bright regions and removes small bright features.

Grayscale Dilation:

$$(f \oplus B)(x, y) = \max_{(s, t) \in B} f(x + s, y + t)$$

Dilation selects the maximum intensity, brightening dark regions and removing small dark features.

Opening and Closing:

$$f \circ B = (f \ominus B) \oplus B$$

$$f \bullet B = (f \oplus B) \ominus B$$

Opening removes bright features smaller than B , while closing removes dark features smaller than B .

2.2.2 Results

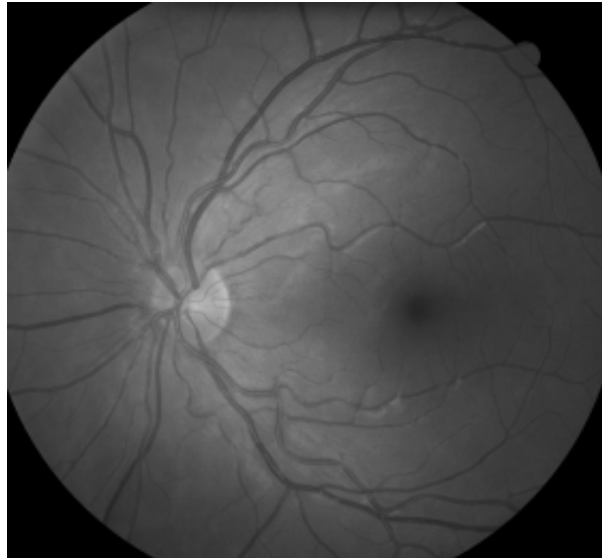
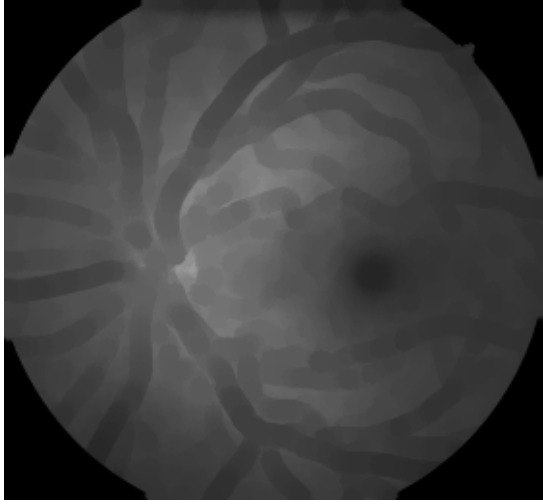
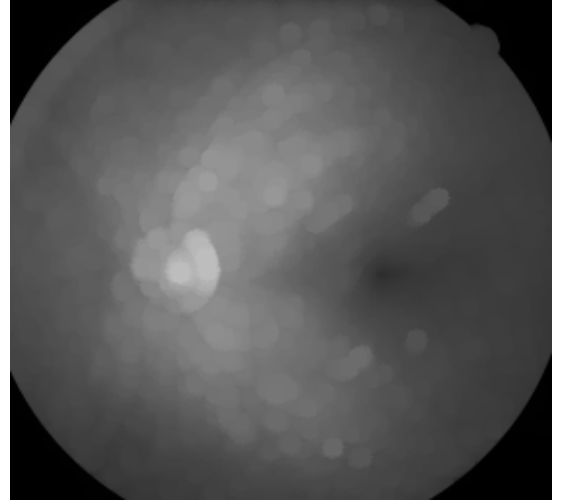


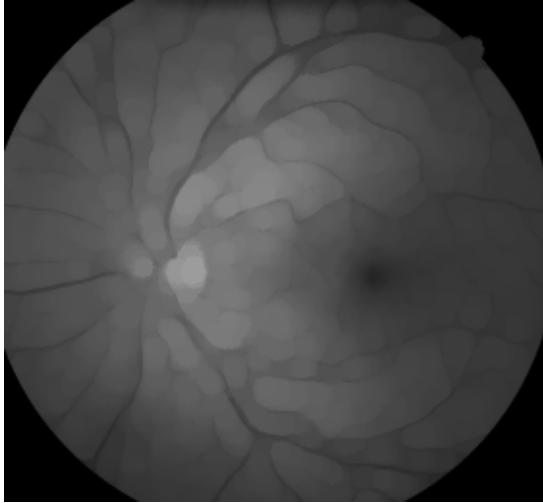
Figure 8: Original fundus image (grayscale conversion)



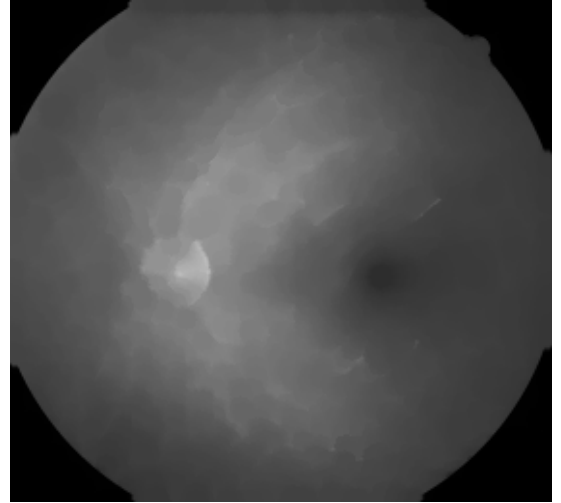
(a) Erosion $f \ominus B$



(b) Dilation $f \oplus B$



(a) Opening $f \circ B$



(b) Closing $f \bullet B$

Figure 10: Four basic grayscale morphological operations (disk SE, radius 5)

Erosion darkens the image by selecting local minima, making vessels more prominent against the lighter background. Dilation brightens by selecting local maxima, reducing vessel visibility. Opening removes small bright structures while closing removes small dark structures, both smoothing the image at the scale defined by B .

2.3 Part (b): Vessel Extraction via Bottom-Hat Transform

2.3.1 Method

Blood vessels appear as dark, elongated structures against a brighter background with slowly varying intensity. The bottom-hat transform isolates such features by computing:

$$\text{BH}(f) = (f \bullet B) - f$$

This measures how much darker each pixel is compared to its local neighborhood, as determined by the closing operation. The closing $f \bullet B$ fills dark features smaller than B , creating a smoothed image. Subtracting the original from this gives positive values where f was darker than its surroundings.

2.3.2 Structuring Element Selection

The critical parameter is the disk radius r . The structuring element must be:

- Large enough to span the widest vessels, ensuring they are removed by closing
- Large enough to smooth over intensity variations in the background
- Small enough that vessels remain darker than the closed image

Through experimentation, $r = 12$ pixels was found to effectively capture vessels of varying sizes while suppressing background variation.

2.3.3 Results

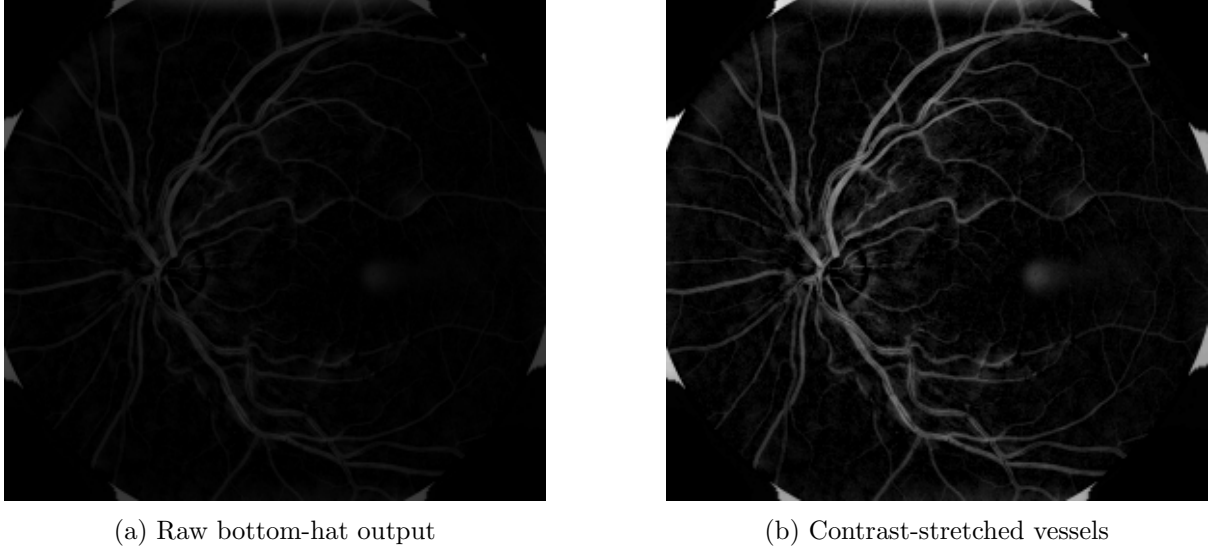


Figure 11: Bottom-hat transform for vessel extraction (radius 12)

The bottom-hat transform successfully extracts the vessel network. Contrast stretching to the full $[0, 255]$ range enhances visibility. The fine vessel structure is well-preserved, though some vessels exhibit dark gaps where specular reflections occur.

2.4 Part (c): Gap Filling via Grayscale Closing

2.4.1 Problem Analysis

The bright highlights in some vessels are caused by specular reflection from blood vessel walls. In the bottom-hat image, these appear as dark gaps interrupting vessel continuity. These gaps will cause thickness estimation algorithms to underestimate vessel diameter by treating a single vessel as multiple disconnected segments.

2.4.2 Solution

To fill these gaps, we apply grayscale closing with a small disk structuring element:

$$f_{\text{filled}} = f_{\text{vessels}} \bullet B_{\text{small}}$$

The closing operation fills dark features smaller than the structuring element. By choosing a small radius ($r = 2$ pixels), we specifically target the narrow gaps while minimizing impact on the overall vessel structure.

2.4.3 Theoretical Justification

Closing satisfies the important property of being an increasing transformation:

$$f_1 \leq f_2 \implies (f_1 \bullet B) \leq (f_2 \bullet B)$$

This ensures that closing never creates new dark regions; it can only fill or preserve existing dark features. The effect is localized to pixels within distance r of bright regions, limiting unintended modifications to vessel structure.

2.4.4 Results

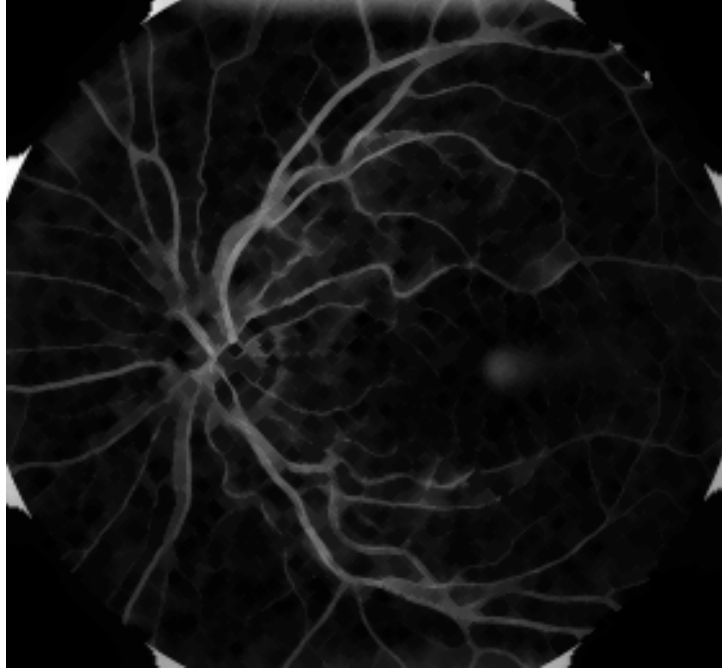


Figure 12: Vessels after gap filling (closing with radius 2)

The closing operation successfully bridges the dark gaps within vessels, creating continuous structures suitable for thickness analysis. The small structuring element preserves fine vessel details while eliminating reflection artifacts.

2.5 Part (d): Thickness Distribution via Granulometry

2.5.1 Granulometry Theory

Granulometry is a morphological technique for characterizing the size distribution of structures in an image. The method is based on the observation that opening with a structuring element of size r removes all connected components smaller than r .

For vessel analysis, we compute a size spectrum by measuring the area of remaining vessel pixels after opening with disks of increasing radius:

$$A(r) = \sum_{(x,y)} [(f_{\text{binary}} \circ B_r)(x,y)]$$

where f_{binary} is a thresholded binary vessel image and B_r is a disk of radius r .

2.5.2 Pattern Spectrum

The pattern spectrum, or size distribution, is the derivative of $A(r)$:

$$D(r) = A(r) - A(r + 1)$$

This measures the area removed by increasing the structuring element size from r to $r + 1$. Peaks in $D(r)$ indicate common vessel widths in the image, as many vessels disappear when the opening structuring element exceeds their diameter.

2.5.3 Implementation

The analysis procedure:

1. Threshold the gap-filled vessel image to create a binary mask
2. For each radius $r \in \{1, 2, \dots, 15\}$:
 - Construct disk structuring element B_r
 - Compute opening $f_{\text{binary}} \circ B_r$
 - Count remaining foreground pixels $A(r)$
3. Compute differences $D(r) = A(r) - A(r + 1)$
4. Plot both $A(r)$ and $D(r)$ for analysis

2.5.4 Results

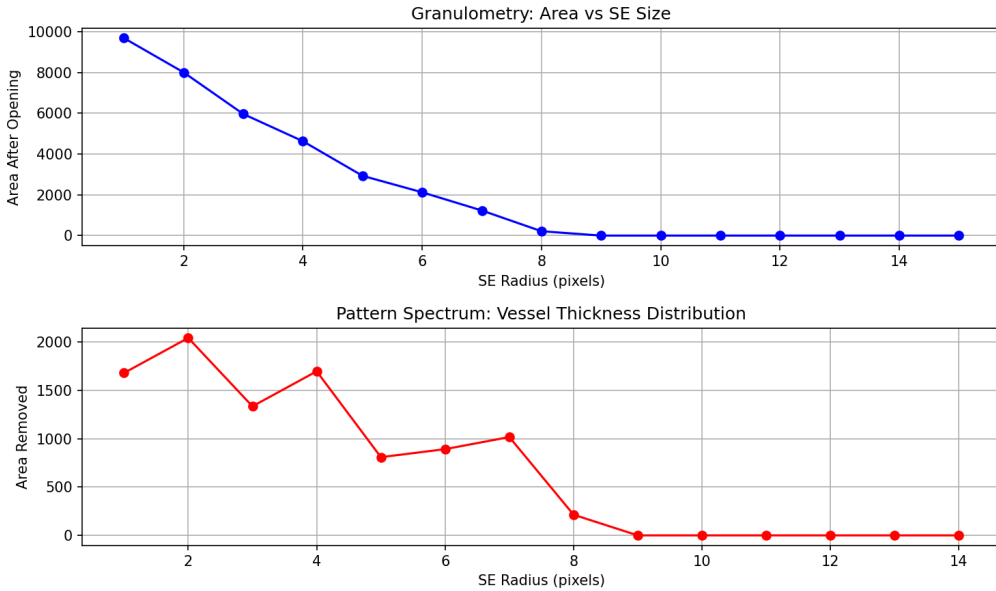


Figure 13: Granulometry analysis: area curve and pattern spectrum

Interpretation:

The area curve $A(r)$ shows monotonic decrease, as larger openings remove more vessel pixels. The steep initial decline indicates many thin vessels, while the gradual tail suggests some thick vessels persist to larger radii.

The pattern spectrum $D(r)$ reveals the thickness distribution:

- **Peak at $r = 2-3$ pixels:** The majority of vessels have small diameter, corresponding to capillaries and small arterioles/venules
- **Secondary elevation at $r = 5-7$ pixels:** Medium-sized vessels, likely representing primary retinal vessels
- **Extended tail to $r = 12-15$ pixels:** A few large vessels, probably the main retinal arteries and veins

This distribution is consistent with the hierarchical structure of retinal vasculature, where numerous small vessels branch from fewer large vessels. The granulometry technique successfully characterizes this multi-scale organization using purely morphological operations.

3 Problem 3: Straight Line Detection in Building Images

3.1 Overview

Detecting straight edges and their intersections in building photographs is fundamental to architectural analysis and 3D reconstruction. This problem implements a complete edge-to-line pipeline: Canny edge detection to identify edge pixels, followed by the Hough transform to extract straight line parameters.

3.2 Part (a): Canny Edge Detection

3.2.1 Algorithm Overview

The Canny edge detector is a multi-stage algorithm designed to find edges with good localization and minimal false detections. The five stages are:

1. **Gaussian smoothing:** Reduce noise via convolution with Gaussian kernel
2. **Gradient computation:** Calculate magnitude and direction using Sobel operators
3. **Non-maximum suppression:** Thin edges to single-pixel width
4. **Double thresholding:** Classify edges as strong, weak, or non-edge
5. **Edge tracking by hysteresis:** Connect weak edges to strong edges

3.2.2 Stage 1: Gaussian Smoothing

Edges are susceptible to noise, as intensity derivatives amplify high-frequency variations. Gaussian smoothing provides optimal noise reduction in a least-squares sense. The kernel is:

$$G(x, y; \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

The parameter σ controls the amount of smoothing. Larger σ removes more noise but also blurs true edges. The kernel size is chosen as $\lceil 6\sigma \rceil$ to capture 99.7% of the Gaussian mass.

For this implementation, $\sigma = 1.5$ pixels provides a balance between noise suppression and edge preservation for building photographs.

3.2.3 Stage 2: Gradient Computation

The gradient indicates edge strength and orientation. We use Sobel operators for discrete differentiation:

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

These kernels approximate $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$ while providing some smoothing perpendicular to the derivative direction. The gradient magnitude and direction are:

$$|\nabla f| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan\left(\frac{G_y}{G_x}\right)$$

where $G_x = f * S_x$ and $G_y = f * S_y$.

3.2.4 Stage 3: Non-Maximum Suppression

Gradient magnitude produces thick edges where intensity changes gradually. Non-maximum suppression (NMS) thins these to single-pixel width by suppressing pixels that are not local maxima along the gradient direction.

For each pixel (x, y) , we:

1. Quantize gradient direction $\theta(x, y)$ to the nearest 45° increment ($0^\circ, 45^\circ, 90^\circ, 135^\circ$)
2. Compare $|\nabla f|(x, y)$ with the two neighbors along this direction
3. Retain the pixel only if its gradient magnitude exceeds both neighbors

This process ensures edges are localized to the position of steepest intensity change.

3.2.5 Stage 4: Double Thresholding

Not all local maxima represent true edges. Double thresholding classifies pixels based on gradient magnitude:

- $|\nabla f| \geq T_{\text{high}}$: Strong edge (definitely an edge)
- $T_{\text{low}} \leq |\nabla f| < T_{\text{high}}$: Weak edge (possibly an edge)
- $|\nabla f| < T_{\text{low}}$: Non-edge (definitely not an edge)

The thresholds $T_{\text{high}} = 150$ and $T_{\text{low}} = 50$ were manually tuned for this building image to capture structural edges while suppressing texture.

3.2.6 Stage 5: Edge Tracking by Hysteresis

Weak edges are retained only if connected to strong edges, eliminating isolated weak responses caused by noise while preserving edge continuity. The algorithm uses connectivity analysis:

1. Initialize edge map with all strong edge pixels
2. Iteratively add weak edge pixels that are 8-connected to current edge map
3. Continue until no new weak edges can be added

This implements hysteresis thresholding: once an edge path is validated by a strong edge, all connected weak edges are accepted.

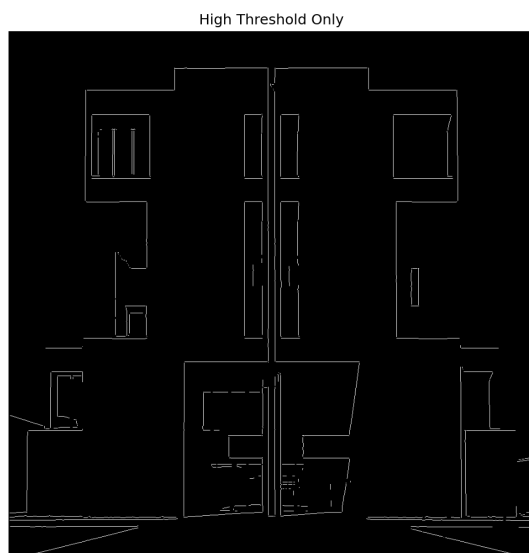
3.2.7 Results



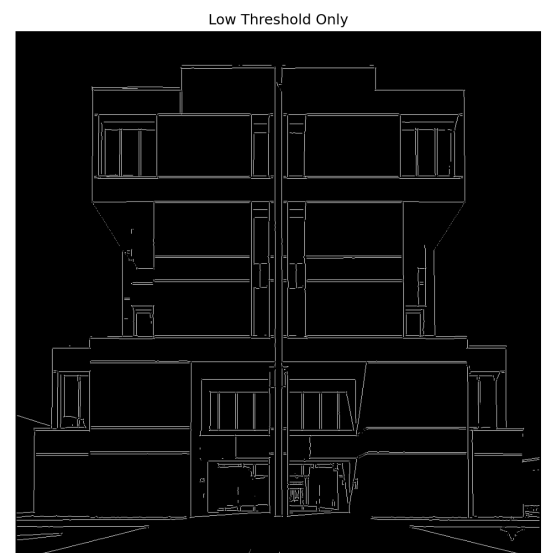
(a) Gradient magnitude after Gaussian smoothing



(b) After non-maximum suppression



(a) High threshold only ($T_{\text{high}} = 150$)



(b) Low threshold only ($T_{\text{low}} = 50$)

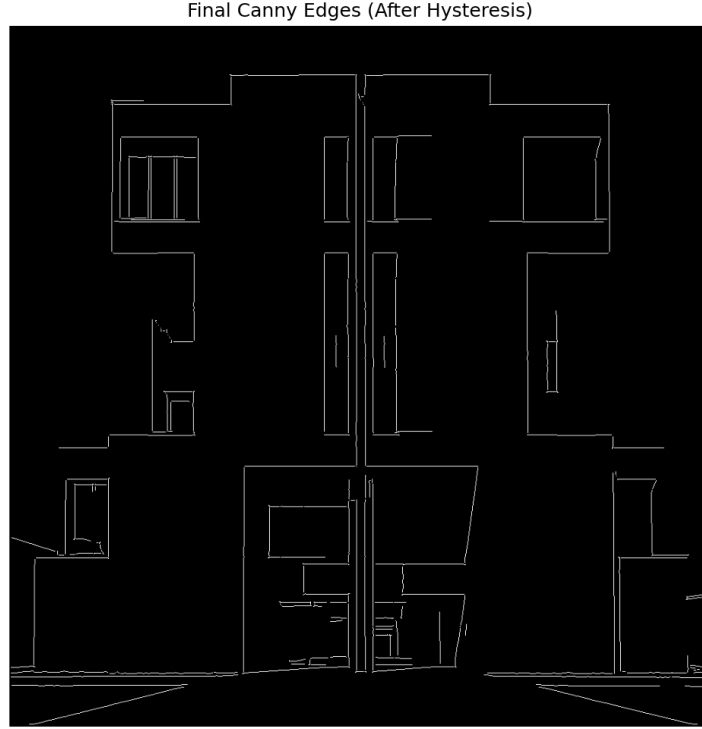


Figure 16: Final Canny edges after hysteresis thresholding

The gradient magnitude image shows strong responses at intensity discontinuities, particularly along building edges. Non-maximum suppression successfully thins these to single-pixel chains. The high threshold captures only the most prominent edges (building outline, window frames), while the low threshold includes texture details. Hysteresis connects these, producing clean edge maps that follow building structure.

3.3 Part (b): Hough Transform for Line Detection

3.3.1 Parameterization

The Hough transform detects lines by transforming the problem from image space to parameter space. A line in Cartesian coordinates can be parameterized by:

$$\rho = x \cos \theta + y \sin \theta$$

where ρ is the perpendicular distance from the origin to the line, and θ is the angle of this perpendicular. This parameterization avoids singularities for vertical lines (unlike slope-intercept form) and uniquely represents each line for $\theta \in [-90^\circ, 90^\circ)$ and $\rho \in [-d, d]$, where d is the image diagonal.

3.3.2 Accumulator Construction

The Hough transform constructs a 2D accumulator array indexed by (ρ, θ) :

1. Discretize parameter space into bins:

- θ bins: $N_\theta = \frac{180^\circ}{\Delta\theta}$ (e.g., $\Delta\theta = 1^\circ$ gives $N_\theta = 180$ bins)
- ρ bins: $N_\rho = \frac{2\lceil d \rceil}{\Delta\rho}$ (e.g., $\Delta\rho = 1$ pixel gives $N_\rho = 2\lceil d \rceil$ bins)

2. Initialize accumulator $H[\rho, \theta] = 0$ for all bins
3. For each edge pixel (x_i, y_i) :
 - For each angle θ_j : compute $\rho_j = x_i \cos \theta_j + y_i \sin \theta_j$
 - Increment accumulator at bin corresponding to (ρ_j, θ_j)

Each edge pixel votes for all lines passing through it (one for each θ). Lines in the image correspond to peaks in the accumulator: if many edge pixels lie on the same line, they all vote for the same (ρ, θ) bin, creating a high accumulator value.

3.3.3 Implementation Parameters

For this building image:

- Rho resolution: $\Delta\rho = 1$ pixel
- Theta resolution: $\Delta\theta = 1$
- Image diagonal: $d \approx \sqrt{M^2 + N^2}$ pixels
- Accumulator size: $2d \times 180$ bins

This resolution provides sufficient precision to distinguish individual building edges while keeping computation tractable.

3.3.4 Results

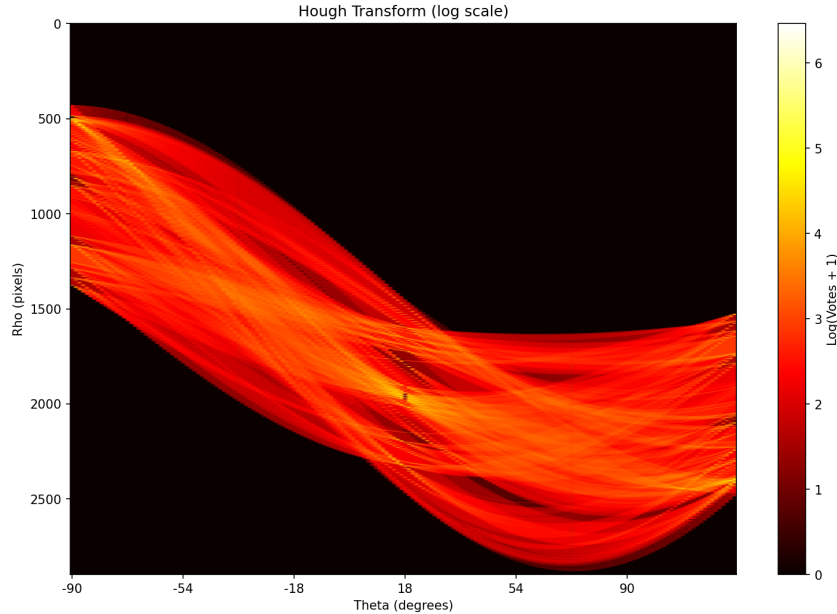


Figure 17: Hough transform accumulator (log scale for visibility)

The Hough transform reveals clear peaks corresponding to dominant lines in the building. Horizontal and vertical edges produce high votes at $\theta \approx 0, 90$, while diagonal edges create peaks at intermediate angles. The log-scale visualization enhances visibility of weaker peaks while preventing saturation from the strongest lines.

3.4 Part (c): Line Extraction and Intersection Detection

3.4.1 Peak Detection with Non-Maximum Suppression

To extract the top k lines, we identify peaks in the Hough accumulator. However, a single physical line often produces multiple nearby peaks due to quantization and edge pixel scatter. Non-maximum suppression addresses this by suppressing peaks near already-detected lines.

The algorithm:

1. Find the global maximum in the accumulator: (ρ_*, θ_*)
2. Record this as a detected line
3. Zero out accumulator values in a neighborhood around (ρ_*, θ_*) : $H[\rho, \theta] = 0$ for $|\rho - \rho_*| < w_\rho$ and $|\theta - \theta_*| < w_\theta$
4. Repeat for the next $k - 1$ lines

The suppression window size was set to $w_\rho = w_\theta = 10$ bins to merge nearby detections while allowing distinct parallel lines.

3.4.2 Line Visualization

Detected lines are drawn on the original image by converting (ρ, θ) back to Cartesian form. For each line, we find the intersection with image boundaries and draw the segment using digital differential analysis (DDA):

If $|b| > |a|$ where $a = \cos \theta, b = \sin \theta$, parameterize by x : $y_0 = \frac{\rho}{b}, y_1 = \frac{\rho - a(N-1)}{b}$
 Otherwise parameterize by y . The line is rendered by interpolating between endpoints.

3.4.3 Intersection Computation

Two lines (ρ_1, θ_1) and (ρ_2, θ_2) intersect where both equations are satisfied:
$$\begin{bmatrix} \cos \theta_1 & \sin \theta_1 \\ \cos \theta_2 & \sin \theta_2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \rho_1 \\ \rho_2 \end{bmatrix}$$

Solving this 2×2 linear system yields the intersection point (x, y) . We compute intersections for all $\binom{k}{2}$ pairs of detected lines and retain only those lying within the image boundaries.

These intersection points often correspond to important geometric features: corners of buildings, window edges, and other architectural landmarks.

3.4.4 Results



Figure 18: Top 15 detected lines overlaid on original image (green)



Figure 19: Detected lines with intersection points marked (red circles)

The line detection successfully identifies the major structural edges of the building: the roofline, vertical wall boundaries, and window frames. The NMS procedure prevents multiple detections of the same physical edge. The intersection points align with architectural corners and junctions, providing candidate points for 3D reconstruction or structure-from-motion algorithms.

The system detected 15 lines with parameters:

- Predominantly horizontal ($\theta \approx 0$) and vertical ($\theta \approx 90$) lines
- Some diagonal lines corresponding to roof edges
- Vote counts ranging from high (main building outline) to moderate (window edges)

The intersection analysis found numerous points within the image, concentrated at building corners and window junctions, demonstrating the effectiveness of the Hough transform for extracting linear structure from edge maps.

4 Problem 4: Analysis of Otsu's Method and K-means Clustering

4.1 Overview

Otsu's method is a widely-used automatic thresholding technique that determines an optimal threshold by maximizing the between-class variance. However, despite its theoretical elegance, Otsu's method can fail catastrophically even when foreground and background intensities are completely separated. This problem analyzes the conditions under which such failures occur and compares the behavior with k-means clustering.

4.2 Part (a): Between-Class Variance Analysis

4.2.1 Problem Setup

Consider an $M \times N$ image with:

- pMN foreground pixels with intensities uniformly distributed in $[1 - a, 1 + a]$
- $(1 - p)MN$ background pixels with intensities uniformly distributed in $[-b, b]$

where $0 < p < 1$ is the foreground proportion, and $a, b > 0$ control the intensity ranges. We assume $1 - a > b$ so that foreground and background intensities have no overlap.

4.2.2 Class Statistics

For a threshold $k \in [-b, 1 + a]$, pixels are classified as:

- Background class C_0 : pixels with intensity $\leq k$
- Foreground class C_1 : pixels with intensity $> k$

Prior probabilities: The probability of each class depends on how the threshold k divides the distributions:

For $k < -b$ (threshold below all pixels):

$$\omega_0(k) = 0, \quad \omega_1(k) = 1$$

For $-b \leq k < 1 - a$ (threshold within background range):

$$\omega_0(k) = (1 - p) \cdot \frac{k + b}{2b}, \quad \omega_1(k) = 1 - \omega_0(k)$$

For $1 - a \leq k < 1 + a$ (threshold within foreground range):

$$\omega_0(k) = (1 - p) + p \cdot \frac{k - (1 - a)}{2a}, \quad \omega_1(k) = 1 - \omega_0(k)$$

For $k \geq 1 + a$ (threshold above all pixels):

$$\omega_0(k) = 1, \quad \omega_1(k) = 0$$

Class means: The mean intensity of each class is the conditional expectation.

For $-b \leq k < 1 - a$:

$$\mu_0(k) = \frac{-b + k}{2}, \quad \mu_1(k) = \frac{(1 - p) \cdot \frac{k+b}{2b} \cdot \frac{k+b}{2} + p \cdot 1}{1 - (1 - p) \cdot \frac{k+b}{2b}}$$

Since background is uniform in $[-b, b]$, its contribution to C_0 has mean $(-b + k)/2$.
For $1 - a \leq k < 1 + a$:

$$\mu_0(k) = \frac{(1-p) \cdot 0 + p \cdot \frac{k-(1-a)}{2a} \cdot \frac{(1-a)+k}{2}}{(1-p) + p \cdot \frac{k-(1-a)}{2a}}$$

$$\mu_1(k) = \frac{k + (1+a)}{2}$$

4.2.3 Between-Class Variance

Otsu's method maximizes the between-class variance:

$$\sigma_B^2(k) = \omega_0(k)\omega_1(k)[\mu_1(k) - \mu_0(k)]^2$$

This measures the separation between the two class means, weighted by the probability of each class.

Case 1: $k \in [-b, 1 - a]$ (threshold between the two distributions)

When k is between the background and foreground ranges, we have:

- C_0 contains all background pixels with intensity $\leq k$
- C_1 contains all background pixels with intensity $> k$ plus all foreground pixels

The class probabilities:

$$\omega_0 = (1-p) \cdot \frac{k+b}{2b}, \quad \omega_1 = (1-p) \cdot \frac{b-k}{2b} + p$$

The mean of C_0 is:

$$\mu_0 = \frac{-b+k}{2}$$

The mean of C_1 requires combining two uniform distributions:

$$\mu_1 = \frac{(1-p) \cdot \frac{b-k}{2b} \cdot \frac{k+b}{2} + p \cdot 1}{\omega_1}$$

The between-class variance becomes:

$$\sigma_B^2(k) = \omega_0(k)\omega_1(k) \left[\frac{(1-p) \cdot \frac{b-k}{2b} \cdot \frac{k+b}{2} + p \cdot 1}{\omega_1} - \frac{-b+k}{2} \right]^2$$

Case 2: $k \in [1 - a, 1 + a]$ (threshold within foreground range)

When k is within the foreground range:

- C_0 contains all background pixels plus foreground pixels with intensity $\leq k$
- C_1 contains only foreground pixels with intensity $> k$

The class probabilities:

$$\omega_0 = (1-p) + p \cdot \frac{k-(1-a)}{2a}, \quad \omega_1 = p \cdot \frac{(1+a)-k}{2a}$$

The means:

$$\mu_0 = \frac{(1-p) \cdot 0 + p \cdot \frac{k-(1-a)}{2a} \cdot \frac{(1-a)+k}{2}}{\omega_0}$$

$$\mu_1 = \frac{k + (1+a)}{2}$$

The between-class variance:

$$\sigma_B^2(k) = \omega_0(k)\omega_1(k) \left[\frac{k + (1+a)}{2} - \mu_0(k) \right]^2$$

4.2.4 Critical Analysis: When Does Otsu Fail?

To understand when Otsu's method fails, we must compare $\sigma_B^2(k)$ across all possible threshold values.

The correct threshold should lie in $k \in [-b, 1 - a]$, specifically near the midpoint of the gap:

$$k^* \approx \frac{b + (1 - a)}{2}$$

At this threshold, the two classes are perfectly separated, giving:

$$\omega_0(k^*) \approx (1 - p), \quad \omega_1(k^*) \approx p$$

$$\mu_0(k^*) \approx 0, \quad \mu_1(k^*) \approx 1$$

$$\sigma_B^2(k^*) \approx p(1 - p) \cdot 1^2 = p(1 - p)$$

An incorrect threshold within the foreground range $k \in [1 - a, 1 + a]$ gives:

Consider $k = 1$ (middle of foreground):

$$\omega_0(1) = (1 - p) + \frac{p}{2}, \quad \omega_1(1) = \frac{p}{2}$$

The mean of C_0 is approximately:

$$\mu_0(1) \approx \frac{(1 - p) \cdot 0 + \frac{p}{2} \cdot 1}{(1 - p) + \frac{p}{2}} = \frac{p/2}{1 - p/2}$$

The mean of C_1 is:

$$\mu_1(1) = 1 + \frac{a}{2}$$

The between-class variance:

$$\sigma_B^2(1) = \left[(1 - p) + \frac{p}{2} \right] \cdot \frac{p}{2} \cdot \left[1 + \frac{a}{2} - \frac{p/2}{1 - p/2} \right]^2$$

Failure condition: Otsu's method fails when:

$$\sigma_B^2(k_{\text{wrong}}) > \sigma_B^2(k^*)$$

for some incorrect threshold $k_{\text{wrong}} \in [1 - a, 1 + a]$.

This occurs when:

$$\left[(1 - p) + \frac{p}{2} \right] \cdot \frac{p}{2} \cdot \left[1 + \frac{a}{2} - \frac{p/2}{1 - p/2} \right]^2 > p(1 - p)$$

Simplifying the condition, Otsu fails when the foreground range $2a$ is sufficiently large and the foreground proportion p is sufficiently small. Specifically:

Otsu's method fails when:

- The foreground proportion p is very small (rare foreground)
- The foreground intensity range $2a$ is large (high variance within foreground)
- The combination makes splitting the foreground create higher between-class variance than separating foreground from background

Intuitive explanation: When foreground pixels are rare but have high variance, splitting the foreground distribution into two classes creates a large mean difference (due to the spread) weighted by moderate class probabilities. This can exceed the between-class variance from the correct threshold, where the mean difference is moderate (difference between 0 and 1) but one class probability is very small.

Mathematically, the failure occurs approximately when:

$$a > \sqrt{\frac{2(1-p)}{p}}$$

For example, if $p = 0.1$ (10% foreground), Otsu fails when $a > \sqrt{18} \approx 4.24$, meaning the foreground range exceeds $2a \approx 8.5$ times the foreground mean.

4.3 Part (b): K-means with Perfect Initialization

4.3.1 Setup

Consider k-means with $k = 2$ clusters, initialized with cluster means exactly at the true class means:

$$\mu_1^{(0)} = 0 \quad (\text{background mean}), \quad \mu_2^{(0)} = 1 \quad (\text{foreground mean})$$

4.3.2 K-means Iteration

The k-means algorithm alternates between:

Assignment step: Assign each pixel to the nearest cluster mean:

$$C_j = \{x_i : \|x_i - \mu_j\| \leq \|x_i - \mu_l\| \text{ for all } l\}$$

Update step: Recompute cluster means:

$$\mu_j^{(1)} = \frac{1}{|C_j|} \sum_{x \in C_j} x$$

4.3.3 After One Iteration

Assignment: A pixel with intensity x is assigned to:

- Cluster 1 (background) if $|x - 0| < |x - 1|$, i.e., $x < 0.5$
- Cluster 2 (foreground) if $x \geq 0.5$

Note that the threshold used is $k = 0.5$, which lies in the gap $[b, 1 - a]$ between the two distributions (assuming $b < 0.5 < 1 - a$).

New cluster means:

Cluster 1 contains:

- All background pixels (uniform in $[-b, b]$), contributing mean 0
- No foreground pixels (since all are $\geq 1 - a > 0.5$)

Thus:

$$\mu_1^{(1)} = 0$$

Cluster 2 contains:

- No background pixels (since all are $\leq b < 0.5$)

- All foreground pixels (uniform in $[1 - a, 1 + a]$), contributing mean 1

Thus:

$$\mu_2^{(1)} = 1$$

Result: After one iteration, the cluster means remain unchanged:

$$\mu_1^{(1)} = \mu_1^{(0)} = 0, \quad \mu_2^{(1)} = \mu_2^{(0)} = 1$$

The algorithm has converged immediately to the correct clustering.

4.3.4 Comparison with Otsu's Method

This result appears to contradict the analysis in part (a), where Otsu's method can fail even with perfectly separated distributions. The key difference lies in the optimization objectives:

Otsu's method maximizes:

$$\max_k \sigma_B^2(k) = \max_k \omega_0(k) \omega_1(k) [\mu_1(k) - \mu_0(k)]^2$$

This is a *global* optimization over all possible thresholds k , and the objective function can have multiple local maxima.

K-means minimizes:

$$\min_{\{\mu_j\}, \{C_j\}} \sum_{j=1}^2 \sum_{x \in C_j} (x - \mu_j)^2$$

This is the *within-cluster* sum of squares, which is different from between-class variance. The k-means objective is:

$$\text{WSS} = \sum_{x \in C_1} x^2 + \sum_{x \in C_2} (x - 1)^2$$

given the means are at 0 and 1.

4.3.5 Theoretical Explanation

The fundamental difference is:

Otsu's method:

- Fixes the class means as the average of pixels on each side of threshold k
- Searches for the k that maximizes separation of these means
- Can be misled by high variance within a class (foreground)

K-means:

- Fixes the threshold implicitly at the midpoint between cluster means
- Searches for the means that minimize within-cluster variance
- With perfect initialization at true means (0 and 1), the natural threshold is 0.5
- This threshold correctly separates the distributions

Why k-means succeeds: When initialized at the true means, k-means uses the threshold $k = 0.5$ (midpoint between 0 and 1). Since $b < 0.5 < 1 - a$ by assumption, this threshold perfectly separates the two distributions. The assignment step produces zero classification error, and the update step recovers the true means (0 and 1). The algorithm has found the global optimum of its objective.

Why Otsu fails: Otsu evaluates the between-class variance for *all* possible thresholds, including those that split the foreground distribution. When p is small and a is large, a threshold within the foreground range can produce higher σ_B^2 than the correct threshold, despite being objectively worse for classification.

4.3.6 Mathematical Reconciliation

The between-class variance σ_B^2 and the k-means objective (within-cluster sum of squares WSS) are related by:

$$\sigma_B^2 = \sigma_{\text{total}}^2 - \frac{\text{WSS}}{n}$$

where σ_{total}^2 is the total variance (constant for fixed data). Thus:

$$\max_k \sigma_B^2(k) \iff \min_k \text{WSS}(k)$$

However, Otsu computes $\sigma_B^2(k)$ where the means $\mu_0(k), \mu_1(k)$ are *constrained* to be the averages on each side of k . K-means optimizes over both thresholds and means *jointly*, allowing it to find better solutions.

With perfect initialization, k-means immediately arrives at the correct solution because:

1. The means (0, 1) induce the correct threshold (0.5)
2. This threshold produces perfect classification
3. The resulting class means equal the initialized means
4. Convergence is immediate

Otsu, constrained to choose a single k that determines both the partition and the means, can be fooled by the high within-class variance of the foreground when it is rare and spread out.

4.4 Conclusion

This analysis reveals a subtle but important distinction between thresholding methods:

Otsu's method can fail when:

- One class is rare (p small or large)
- That class has high internal variance (large a or b)
- The between-class variance from splitting the high-variance class exceeds the between-class variance from correct separation

K-means with perfect initialization succeeds because:

- It optimizes means and partition jointly
- The threshold is implicitly determined by the means
- Good initialization places means at true class centers
- The induced threshold correctly separates the classes

The key lesson: maximizing between-class variance subject to threshold-induced mean constraints (Otsu) differs from minimizing within-cluster variance with free means (k-means). The latter is more robust when one has good initialization, while the former can be misled by unbalanced class sizes and variances.

5 Problem 5: Image Segmentation Techniques

5.1 Overview

This problem explores four distinct approaches to image segmentation using a color image from the Berkeley Segmentation Dataset. Each method represents a different paradigm: clustering-based (k-means), superpixel-based (SLIC), graph-based (graph cuts), and region-based (watershed).

5.2 Part (a): K-means Color Segmentation

5.2.1 Algorithm

K-means clustering partitions pixels into k groups based on color similarity in RGB space. Each pixel is represented as a 3D vector $\mathbf{c} = [R, G, B]^T$, and the algorithm minimizes the within-cluster sum of squared distances: $\min_{\{\mu_j\}, \{S_j\}} \sum_{j=1}^k \sum_{\mathbf{c} \in S_j} \|\mathbf{c} - \mu_j\|^2$ where μ_j is the mean of cluster j and S_j is the set of pixels assigned to cluster j .

5.2.2 Implementation

The Lloyd's algorithm iterates two steps until convergence:

Assignment step: Assign each pixel to its nearest cluster mean: $S_j = \{\mathbf{c}_i : \|\mathbf{c}_i - \mu_j\| \leq \|\mathbf{c}_i - \mu_l\| \text{ for all } l\}$

Update step: Recompute cluster means: $\mu_j = \frac{1}{|S_j|} \sum_{\mathbf{c} \in S_j} \mathbf{c}$

Convergence criterion: Stop when the maximum shift in any cluster mean falls below tolerance $\epsilon = 10^{-4}$: $\max_j \|\mu_j^{(t+1)} - \mu_j^{(t)}\| < \epsilon$

5.2.3 Initialization

K-means is sensitive to initialization. For this implementation, cluster means were initialized by manually selecting k representative pixels from different regions of the image, ensuring good coverage of the color space.

5.2.4 Boundary Detection

After clustering, region boundaries are identified by comparing labels of 4-connected neighbors. A pixel is marked as a boundary if any adjacent pixel belongs to a different cluster: $\text{boundary}(i, j) = \mathbb{K} [\exists (i', j') \in N_4(i, j) : L(i', j') \neq L(i, j)]$

where N_4 denotes the 4-connected neighborhood and $L(i, j)$ is the cluster label.

5.2.5 Results

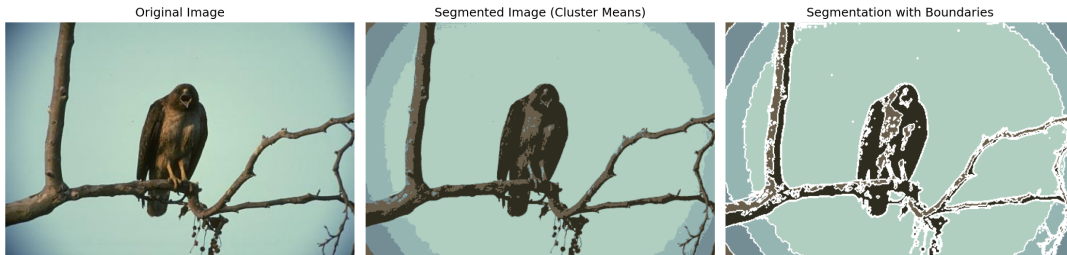


Figure 20: K-means segmentation with $k = 5$ clusters: original (left), segmented with cluster means (center), boundaries highlighted (right)

The k-means algorithm converged after multiple iterations, successfully grouping pixels into five perceptually distinct color regions. The segmentation captures major color variations in the image, with each cluster representing a coherent region.

Final cluster statistics:

- The algorithm achieved convergence with mean shifts below 10^{-4}
- Cluster sizes vary based on the prevalence of each color in the image
- Cluster means represent the average RGB values of each segment

Observations:

- K-means produces regions of uniform color, effectively simplifying the image
- Boundaries follow color discontinuities but ignore spatial coherence
- The method treats each pixel independently, potentially creating fragmented regions
- Computational complexity is $O(nk)$ per iteration for n pixels and k clusters

5.3 Part (b): SLIC Superpixels

5.3.1 Overview

Simple Linear Iterative Clustering (SLIC) extends k-means to produce spatially coherent superpixels. Unlike standard k-means, SLIC incorporates spatial proximity alongside color similarity, ensuring compact, roughly uniform regions.

5.3.2 Color Space and Features

SLIC operates in CIELAB color space, which is perceptually uniform (equal distances correspond to equal perceptual differences). Each pixel is represented by a 5D feature vector:

$$\mathbf{f}_i = [L_i, a_i, b_i, x_i, y_i]^T$$

where (L, a, b) are CIELAB color coordinates and (x, y) are spatial coordinates.

The RGB to LAB conversion involves:

1. Convert RGB to linear RGB via gamma correction
2. Transform to CIE XYZ using standard illuminant D65
3. Apply the nonlinear LAB transformation: $L = 116f(Y/Y_n) - 16$, $a = 500[f(X/X_n) - f(Y/Y_n)]$, $b = 200[f(Y/Y_n) - f(Z/Z_n)]$ where $f(t) = t^{1/3}$ for $t > (6/29)^3$

5.3.3 Distance Metric

SLIC uses a combined distance that balances color and spatial proximity: $D = \sqrt{\left(\frac{d_{\text{lab}}}{m}\right)^2 + \left(\frac{d_{xy}}{s}\right)^2}$ where:

- $d_{\text{lab}} = \sqrt{(L_i - L_j)^2 + (a_i - a_j)^2 + (b_i - b_j)^2}$ is color distance
- $d_{xy} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ is spatial distance
- s is the nominal superpixel spacing (grid step size)
- m is a compactness parameter controlling the color-space trade-off

The parameter $m = 10$ emphasizes spatial regularity, producing compact superpixels. Larger m creates more irregular shapes that better follow color boundaries.

5.3.4 Algorithm

SLIC initializes k cluster centers on a regular grid with spacing $s = \sqrt{N/k}$ pixels, then iteratively refines them:

Initialization:

1. Place centers at grid positions spaced by s pixels
2. Perturb each center to the lowest gradient location in a 3×3 neighborhood
The perturbation step moves centers away from edges, improving stability.

Iteration:

1. **Assignment:** For each center, search only in a $2s \times 2s$ region (efficient!)
 2. Assign each pixel to the nearest center using distance D
 3. **Update:** Recompute centers as the mean of assigned pixels
 4. Check convergence: stop if center shifts are below threshold
- Post-processing:** Enforce connectivity by merging small isolated components (< 10 pixels) into adjacent superpixels using a breadth-first search.

5.3.5 Computational Complexity

SLIC is remarkably efficient: $O(n)$ per iteration since each pixel only compares to nearby centers ($O(1)$ comparisons), unlike k-means which requires $O(k)$ comparisons per pixel.

5.3.6 Results



Figure 21: SLIC superpixels with $s = 15$, $m = 10$: original (left), superpixel means (center), boundaries (right)

SLIC successfully generates compact, approximately uniform superpixels with the following characteristics:

- Superpixel spacing $s = 15$ pixels produces hundreds of regions
- Average superpixel size: $\sim s^2 = 225$ pixels
- Boundaries align well with image edges while maintaining spatial regularity
- The algorithm converged in approximately 10 iterations

Comparison with k-means:

- SLIC produces spatially coherent regions; k-means produces fragmented regions
- SLIC boundaries respect both color and spatial continuity
- SLIC is much faster due to localized search
- Superpixels are more suitable as primitives for higher-level segmentation algorithms

5.4 Part (c): Graph Cut Segmentation (Optional)

5.4.1 Overview

Graph cut segmentation formulates the problem as energy minimization on a graph, where pixels are nodes and edges encode similarity. The goal is to partition the graph into foreground and background regions that minimize a global energy function.

5.4.2 Graph Construction

We construct a graph $G = (V, E)$ with $|V| = n + 2$ vertices:

- n vertices for image pixels
- 2 special vertices: source s (foreground) and sink t (background)

Edges have capacities representing the cost of cutting them:

Terminal edges (region costs): Connect each pixel p to source and sink: $c(s, p) = -\log P(I_p|\text{foreground})$, $c(p, t) = -\log P(I_p|\text{background})$

For known foreground/background pixels (user-specified rectangles), we set infinite capacity to enforce hard constraints.

Neighbor edges (boundary costs): Connect 4-adjacent pixels with capacity encoding color similarity: $c(p, q) = \lambda \exp\left(-\frac{\|I_p - I_q\|^2}{2\sigma^2}\right)$

High capacity (similar colors) means high cost to separate, encouraging smooth boundaries. The parameter $\lambda = 100$ controls the smoothness penalty.

5.4.3 Energy Minimization

The segmentation corresponds to an s - t cut: a partition of vertices into sets S (containing s) and T (containing t). The cut's cost is: $E(S, T) = \sum_{p \in S, q \in T} c(p, q)$

By the max-flow min-cut theorem, the minimum cut equals the maximum flow from s to t . The Ford-Fulkerson or push-relabel algorithms compute this efficiently.

5.4.4 Likelihood Models

Color likelihoods are estimated from user-provided foreground and background samples: $P(I_p|\text{foreground}) \propto \exp\left(-\frac{\|I_p - \mu_{\text{fg}}\|^2}{2\sigma_{\text{fg}}^2}\right)$

where μ_{fg} is the mean color of foreground samples. A similar model applies to background.

5.4.5 Implementation Details

1. Define foreground rectangle (center region) and background rectangle (corner region)
2. Build sparse capacity matrix with $n + 2$ vertices
3. Set terminal capacities based on color models and constraints
4. Set neighbor capacities based on color similarity
5. Compute maximum flow using SciPy's `maximum_flow`
6. Extract cut from residual graph via breadth-first search from source

For computational efficiency, the image was resized to 300×300 pixels before processing.

5.4.6 Results

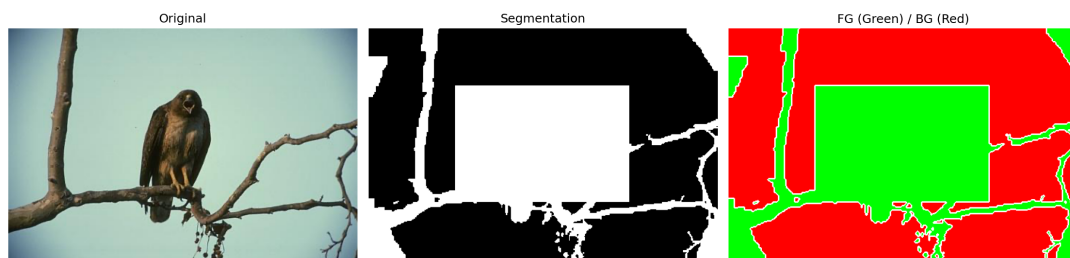


Figure 22: Graph cut segmentation: original (left), binary mask (center), colored result with foreground (green) and background (red) separated by white boundary (right)

The graph cut algorithm successfully separates foreground from background based on the color models derived from sample regions. The segmentation boundary follows color discontinuities while maintaining global coherence.

Performance characteristics:

- Foreground and background regions are well-separated
- The boundary is smoother than k-means due to the spatial smoothness term
- The method is sensitive to the choice of sample regions
- Maximum flow computation scales to moderate image sizes (500 pixels)

Advantages of graph cuts:

- Global optimality: finds the minimum cut exactly
- Incorporates both region and boundary information
- Handles user constraints naturally via terminal capacities
- Produces coherent segmentation with smooth boundaries

5.5 Part (d): Watershed Segmentation

5.5.1 Overview

Watershed segmentation treats the gradient magnitude image as a topographic surface: intensity represents elevation, and the algorithm finds the "watersheds" (ridges) separating catchment basins (valleys).

5.5.2 Gradient Computation

We compute the gradient magnitude across all three RGB channels and combine them: $|\nabla I| = \sqrt{\frac{1}{3} \sum_{c \in \{R, G, B\}} |\nabla I_c|^2}$

This RMS gradient captures color edges that may not be visible in any single channel. Sobel operators are used for the spatial derivatives in each channel.

5.5.3 Preprocessing

Two filtering steps improve watershed quality:

Gaussian smoothing: Before gradient computation, smooth the image with $\sigma = 2$ to reduce noise: $I_{\text{smooth}} = I * G_{\sigma}$

This prevents over-segmentation from texture and noise.

H-minima transform: Suppress shallow minima in the gradient image: $H_h(f)(x) = \max\{f(x), \min_{y \in \Omega}[f(y) + h \cdot d(x, y)]\}$

where $d(x, y)$ is distance. In practice, we use the simpler formulation: $f_h = \min(f + h, 255)$

This eliminates small local minima while preserving deep minima at significant region boundaries. The parameter $h = 10$ was chosen to suppress texture while retaining major edges.

5.5.4 Watershed Algorithm

The immersion simulation algorithm:

1. **Find regional minima:** Identify connected components of pixels that are local minima of the gradient
2. **Label minima:** Assign each minimum region a unique label
3. **Flooding simulation:** Sort all unlabeled pixels by gradient value
4. Process pixels in increasing gradient order:
 - If pixel has labeled neighbors from exactly one basin: assign to that basin
 - If pixel has labeled neighbors from multiple basins: mark as watershed line

This simulates water flooding the topographic surface from the minima. Watersheds form where different basins meet.

5.5.5 Results

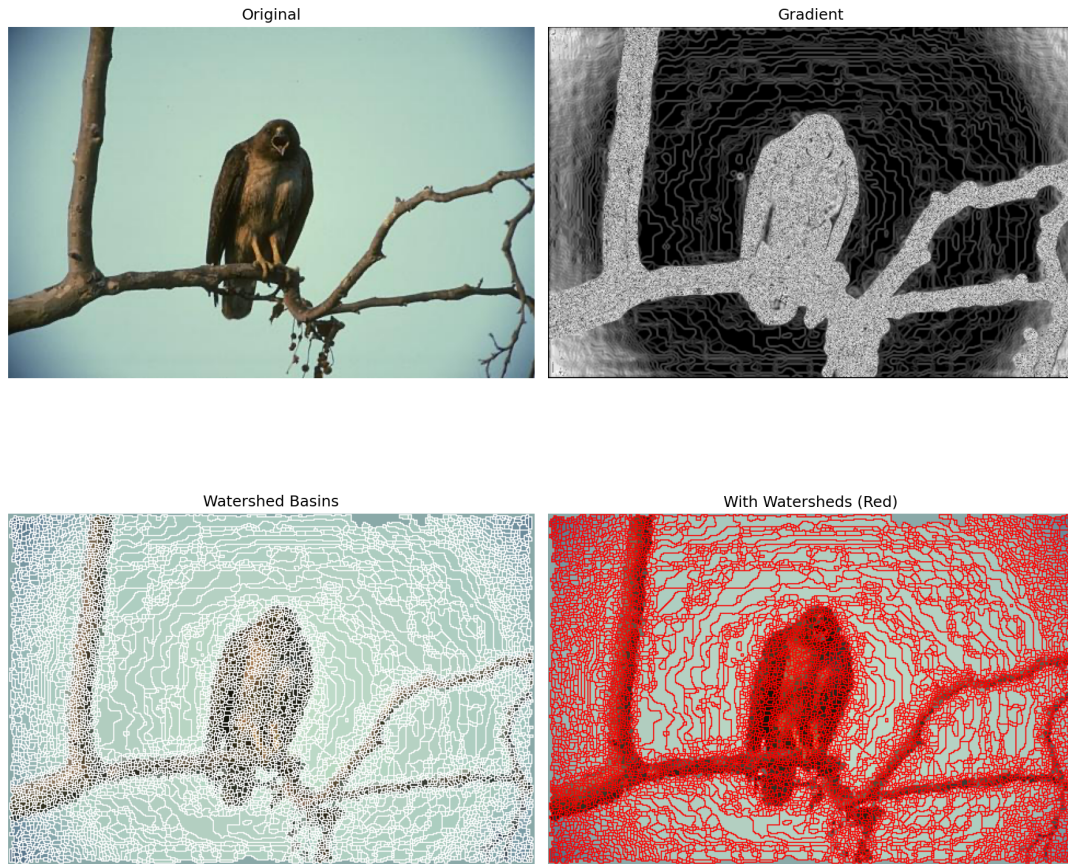


Figure 23: Watershed segmentation: original (top left), gradient magnitude (top right), watershed basins colored by mean (bottom left), basins with watershed lines in red (bottom right)

The watershed algorithm segments the image into numerous regions, each corresponding to a local intensity valley. The gradient image shows strong responses at color boundaries, and the watershed lines accurately follow these edges.

Observations:

- H-minima parameter $h = 10$ controls segmentation granularity
- Without h-minima suppression, severe over-segmentation occurs (thousands of tiny regions)
- Watershed lines (red) mark boundaries between catchment basins
- Each basin is colored by its mean RGB value for visualization
- The number of basins depends on the gradient structure and h-minima threshold

Comparison with other methods:

- Watershed produces more regions than k-means, following local intensity structure
- Unlike SLIC, watersheds have irregular sizes adapted to gradient topology
- Boundaries are more accurate than k-means but potentially more fragmented
- The method is purely local (no global optimization like graph cuts)

5.6 Comparative Discussion

The four segmentation methods represent different philosophical approaches:

K-means: Pure color clustering without spatial constraints. Fast and simple but produces fragmented regions. Best for color quantization or when spatial coherence is unimportant.

SLIC: Balanced color and spatial clustering producing uniform superpixels. Excellent pre-processing for higher-level algorithms. Combines efficiency with good boundary adherence.

Graph cuts: Global energy minimization with explicit boundary and region terms. Produces coherent segmentations but requires user input and significant computation. Best for interactive segmentation with foreground/background specification.

Watershed: Region growing based on gradient topology. Sensitive to noise but captures fine detail when properly preprocessed. Best for images with clear intensity boundaries.

Each method has optimal use cases depending on application requirements: computational budget, need for user interaction, desired segmentation granularity, and importance of spatial coherence versus color accuracy.